

Veridict

ZK-attested spec-driven code review
AI writes the code. Math approves the merge.

Sumit Vekariya (solo entry)
Secure Program Synthesis Hackathon, May 22–24, 2026
Tracks 2 (Specification Validation) and 3 (Spec-Driven Development & Evaluation)
Code: <https://github.com/Zkred/veridict>
Live: <https://veridict.zkred.tech>

Abstract

Most code review today is a single-point-of-trust system: one named maintainer approves a merge, and the project’s security rests on that identity not being compromised, harassed, or fooled. AI synthesis makes this worse, because review volume now exceeds what any one reviewer can verify by reading. Veridict is a merge gate that decouples *qualified-reviewer-ness* from *identity*. A natural-language specification is sent to Claude, which produces an implementation, a pytest suite, and a Z3 invariant file. An issuer runs all three formal layers against the synthesised PR and only mints a Longfellow ZK credential if every layer passes. Reviewers prove credential ownership anonymously; N proofs flip a GitHub branch-protection commit status, unlocking merge. Built end-to-end on a real public repository with active branch protection. The core primitive (Longfellow) is the same one Google designed for anonymous age verification on mobile driver’s licences, repurposed here for code review credentials.

* * *

1. Introduction

“We have no way of knowing if what we’re vibecoding does what we think it does.” –
hackathon brief

Open-source review is leaking trust along three independent failure modes. Each one shows up in a different recent incident, in projects that have nothing else in common:

bincode, December 2024. A widely-used Rust serialisation crate. The maintainer rewrote git history and disabled the issue tracker while migrating from GitHub to SourceHut. The pattern matched supply-chain attack signatures (xz-utils / Jia Tan, March 2024), so the community publicly tried to investigate. Discussion escalated into doxxing and home-address publication. The maintainer shut the project down permanently. There is no bincode 1.3.4.

xz-utils, March 2024. “Jia Tan” spent two years building maintainer credibility on the xz-utils project, then inserted a backdoor that nearly compromised sshd on every major Linux distribution. Caught essentially by accident by Andres Freund at Microsoft (Valgrind anomalies during unrelated benchmarking), weeks before the backdoor would have shipped in stable distro releases.

curl, January 2024. Daniel Stenberg, curl’s maintainer, wrote a public blog post titled “*The I in LLM stands for intelligence,*” describing how curl’s HackerOne reports now skew toward LLM hallucinations. “*My job is increasingly fact-checking the AI.*” The pattern: AI generates plausible-looking PRs and bug reports that pass surface checks and fail on invariants.

Vibe coding, February 2025. Andrej Karpathy coined “vibe coding”: writing code by prompting an LLM and shipping when it *looks* right. The term went viral. The practice became the default workflow on many teams within weeks.

These are different incidents. They share a shape:

- **Identity-coupled trust at the centre.** Trust is bound to a named individual, so attacking the individual (doxxing, social engineering, account takeover, or simply earning their trust over years) compromises the project.
- **No formal gate before human approval.** Reviewers rely on type-checks and “looks right” reading. Subtle invariant violations slip through.
- **Reviewers pay the cost** in burnout, doxxing, or undetected backdoors.

The hackathon brief asks for *spec-driven development and validation* (Tracks 2 and 3). Veridict is one architectural response: replace the named maintainer with N anonymous credentialed reviewers, and replace “looks right” with three formal layers (mypy + pytest + Z3) that the issuer enforces before any reviewer can sign.

* * *

2. Related Work

Longfellow-ZK. Google’s recent ZK system for proving facts about ISO 18013-5 MDOC credentials (mobile driver’s licences) without disclosing the credential itself. The motivating use case is age verification: prove “I’m over 18” without showing your name or date of birth. Longfellow’s contribution over earlier schemes (e.g. BBS+) is operational: it works with the ECDSA signatures already on mobile mDLs and existing issuer infrastructure, without protocol changes. The paper’s abstract claims it is “*the first one that can be deployed without changing any issuer processes, without requiring changes to mobile devices, and without requiring non-standard cryptographic assumptions.*” Proof generation is sub-second on a phone. Veridict adopts Longfellow wholesale as the credential layer, applying a small patch to register an `org.example.viewer` namespace.

Vericoding / spec-driven program synthesis. A growing literature on generating implementations from specifications using LLMs combined with formal verification feedback (CEGIS-style loops, refinement, Lean/Dafny-backed validation). Veridict’s synthesis pipeline (Claude → implementation + tests + Z3 invariants → verify → accept or reject) is a lightweight, hackathon-scale instance of this pattern. The novelty is wiring the verification result into a *credential-issuance gate* rather than just a CI status.

Anonymous code review. Prior work on anonymising code review (e.g. blinded peer review for academic-style code review) exists in research settings, but is not deployed in production OSS workflows. Veridict’s contribution is the threat-model framing (anonymous toward the verifier, not toward the issuer) and the integration with GitHub’s existing branch-protection primitive.

Z3 for specification checking. Z3 is the industry-standard SMT solver. Veridict uses it in a constrained mode: the synthesis prompt instructs Claude to emit a `z3_*.py` file that asserts

the negation of each spec invariant is unsat. This is symbolic verification of the specification’s *consistency and core invariants*, not a full functional refinement proof of the implementation. We are explicit about that scope.

GitHub Apps and branch protection. GitHub’s commit-status API combined with required-status branch protection rules is the deployment substrate. Verdict’s bot pushes a zk-review-gate status that the protection rule treats as a required check. This means GitHub itself enforces the gate; Verdict cannot be bypassed by a maintainer “force-approving” through the UI.

* * *

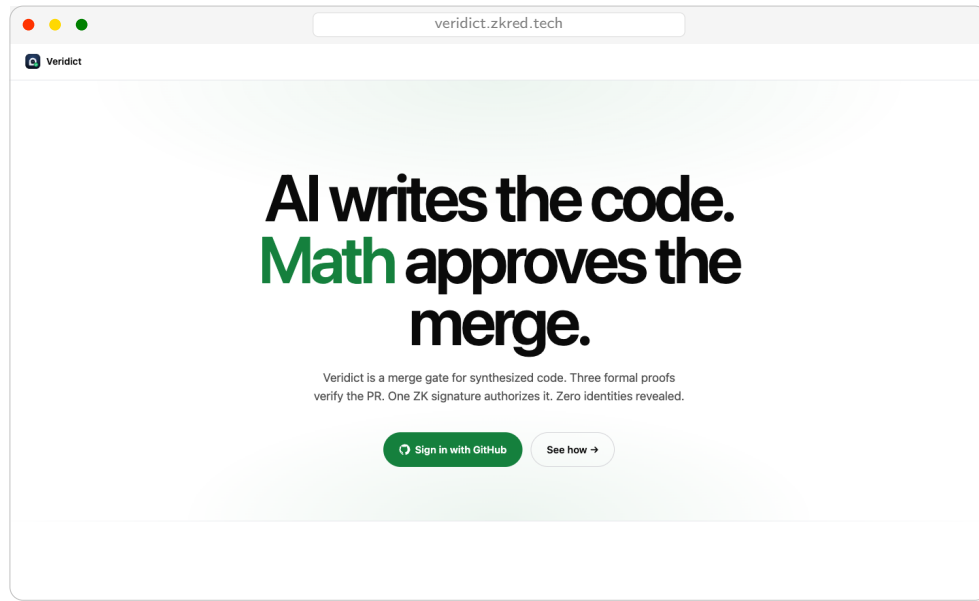


Figure 1: Verdict landing page. The product framing leads with the architectural claim: AI writes the code; formal verification and a ZK proof, not a human signature, authorize the merge.

3. Methodology

3.1 System architecture

Trust roots. Two distinct GitHub identities are deliberately split:

Role	Identity	Scope
Reviewer authentication	GitHub OAuth App “Anonymous Review”	read:org
Bot actions (comments, statuses, synth PRs)	GitHub App “anonymous-review-bot”	contents:write, pull_requests:write, commit_statuses:write, issues:write

The OAuth App can read who you are. The GitHub App writes to the repo. They never overlap. The reviewer’s OAuth identity stays inside the issuer; the bot identity is the only thing GitHub sees on the PR.

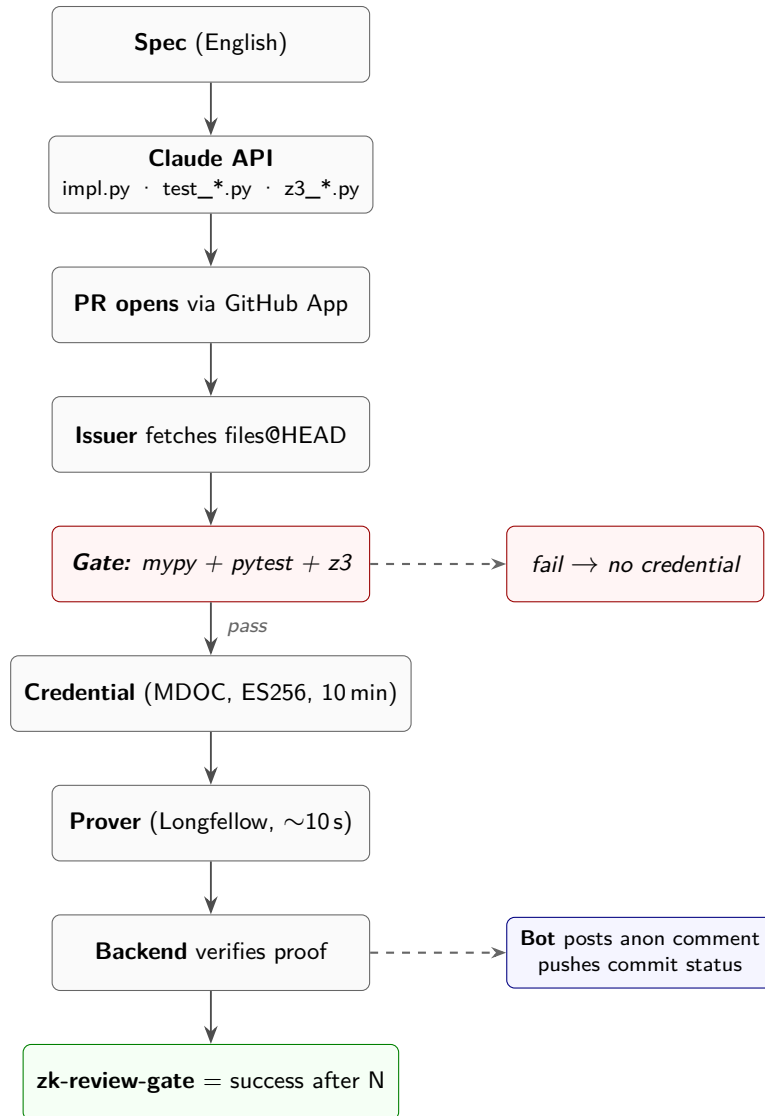


Figure 2: Verdict pipeline. The red gate is the only thing that can block a credential from being minted; everything downstream is either cryptographically bound to that credential (prover, backend) or driven by the verified proof (bot comment, commit status, merge unlock).

3.2 The synthesis pipeline (Tracks 2 + 3)

Claude is given a single-paragraph natural-language specification and a structured-output prompt that demands exactly three files: an implementation, a pytest suite, and a Z3 invariant file. The prompt enforces:

- All tests use *valid* inputs that satisfy preconditions, except tests explicitly named `test_*invalid*` or `test_*raises*` (which use `pytest.raises`).
- Numeric assertions use `pytest.approx` rather than naked equality.
- The Z3 file starts exactly with `from z3 import *` (mandatory, because z3 symbols are only available via wildcard import), defines the spec's data model symbolically, asserts each invariant's negation is unsat, and ends with `print("All Z3 properties verified.")`.

These rules were added after observing failure modes during iteration: Claude was emitting tests

with `rate=0` while the implementation raised on `rate <= 0`, and Z3 files were qualifying symbols as `z3.Solver` (a common mistake when only the `Solver` class is in scope from `from z3 import *`).

3.3 The verification gate

`issuer/spec_checker.py` fetches every Python file in the PR from `raw.githubusercontent.com` at the head SHA, writes them to a temporary directory, and runs three subprocesses:

1. **mypy** with `--ignore-missing-imports`, only on implementation files (we explicitly skip `test_*` and `z3_*` files because they wildcard-import and would emit false positive name-undefined errors).
2. **pytest** with `--tb=short -q`, all files in scope.
3. **Z3 invariant files** invoked directly (`python z3_*.py`), expected to print "All Z3 properties verified." on success.

If any of the three exits non-zero, the issuer refuses to mint a credential. The check is also re-run server-side at approve time; a tampered client cannot spoof it.

3.4 The credential

The MDOC is a CBOR document conforming to ISO 18013-5, with:

- A `validityInfo` window of 10 minutes from issuance (`signed`, `validFrom`, `validUntil`).
- A `nameSpaces` map containing the reviewer's role as a CBOR `text(10)` "maintainer" or `text(8)` "reviewer", under the `org.example.reviewer` namespace.
- A `valueDigests` map mapping each attribute element ID to its SHA-256 digest.
- A `deviceKey` populated from a fresh per-session ECDSA P-256 keypair, encoded as a standard COSE_Key for EC2 (labels `-1: crv`, `-2: x`, `-3: y`; *not* the `dev_key_pfx_ / dev_key_pky_` labels that the variable names in Longfellow's parser misleadingly suggest).

The MDOC is signed by the issuer's ES256 key. A reviewer-side device signature is produced inside the prover.

3.5 The ZK proof and anti-replay binding

Each proof is bound to a session transcript derived from the PR key:

```
pr_key = "{owner}/{repo}/{pr_number}/{commit_sha}"
handover = ["AnonReviewv1", sha256(pr_key)]
transcript = cbor2.dumps([None, None, handover])
```

The byte-identity of this transcript hex string between prover and verifier is critical; the same encoding must be used on both sides. A proof minted for PR A cannot be replayed on PR B, and a force-push that changes the commit SHA invalidates all prior approvals.

The Longfellow prover takes ~10 seconds (most of which is one-time circuit compilation). The verifier takes ~7 seconds. Proof size is ~360 KB. The verifier needs ~365 MB peak RSS, which is why the backend runs on Cloud Run with 4 GB rather than a typical free-tier 512 MB box.

3.6 Pseudonyms and reputation chips

A reviewer’s per-PR pseudonym is `HMAC-SHA256(server_secret, user_id || pr_key)` truncated to 6 hex characters. Same reviewer + same PR → same tag; different PR → unlinkable tag. The HMAC secret lives at `.secrets/pseudonym-key.bin` with mode `0600`.

The bot’s anonymous comment carries the pseudonym, a deterministic identicon (DiceBear via `wsrv.nl` proxy with theme-aware background), and a row of “reputation chips” bucketed from the reviewer’s OAuth profile: role (maintainer/reviewer), account age (5y+ on GitHub), public repo count, and the spec-check result (mypy+pytest+z3 OK). The buckets are coarse on purpose, to reveal trust signal without enabling identity triangulation.

3.7 What is NEW work in the hackathon

New in hackathon	Existing work used
Spec-check gate (mypy + pytest + Z3) wired to credential issuance	mypy, pytest, Z3 (used individually before)
Claude synthesis prompt that emits impl + tests + Z3 invariants in one structured response	Claude API (Anthropic), structured output patterns
Reviewer-namespace patch on Longfellow (patches/add-reviewer-namespace.patch)	Longfellow-ZK (Google)
MDOC builder for reviewer credentials, with custom namespace and 10-minute validity window	cbor2, COSE_Sign1 conventions, ISO 18013-5
ZK proof flow integrated with GitHub branch protection via commit-status API	GitHub Apps, branch protection rules
Pseudonym + reputation-chip system with HMAC-derived stable per-PR identifier	DiceBear identicons, <code>wsrv.nl</code> proxy
Full UX (landing page, dashboard, two-column review page, animated pipeline, dark+light themes)	—

* * *

4. Results

A live demo is deployed on Google Cloud Run with a real public GitHub repository (`vayunetwork/anonymous-review-demo`) configured with branch protection requiring the `zk-review-gate` context to flip success before `main` accepts a merge.

4.1 End-to-end run

1. Sign in to `https://veridict.zkred.tech` with GitHub OAuth.
2. Click *Synthesize with AI*, write a spec (e.g. *“Implement a token-bucket rate limiter with capacity and rate parameters; rate must be positive.”*).

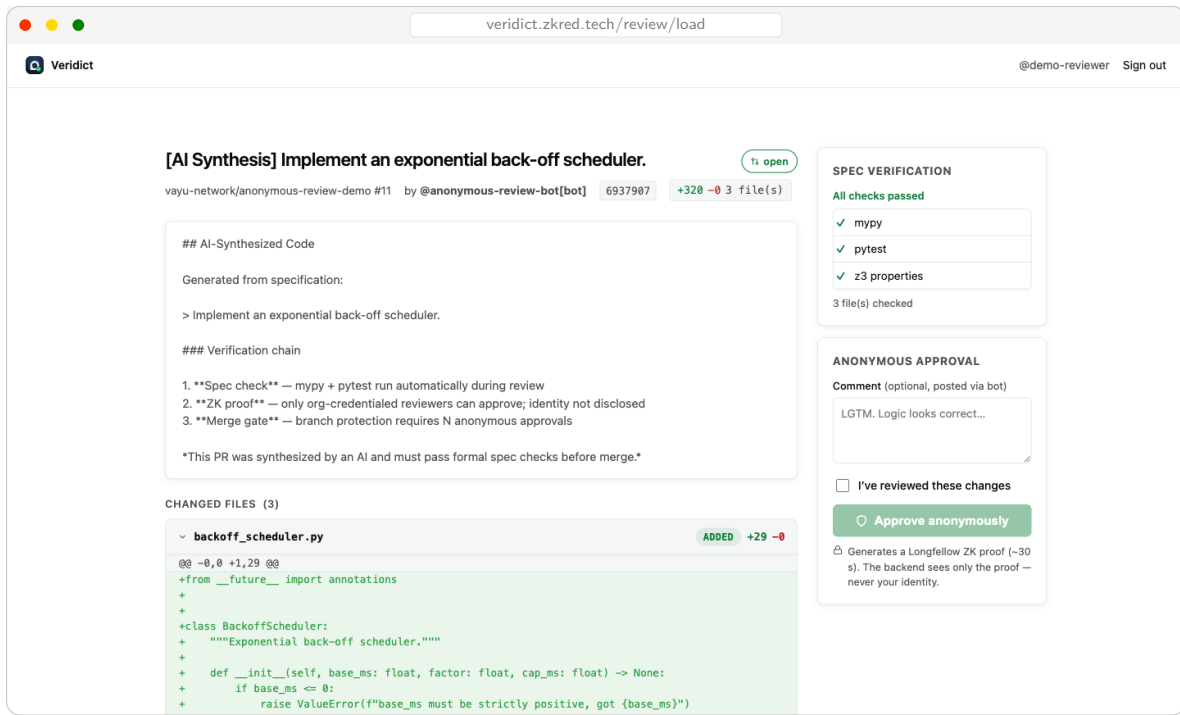


Figure 3: The reviewer’s view of a synthesised PR. Left column: PR title, metadata, full diff (collapsible per file). Right sidebar (sticky): all three spec-check layers passing, comment box, and the disabled-until-checked approve button.

3. Claude generates `token_bucket.py`, `test_token_bucket.py`, `z3_token_bucket.py`. The GitHub App opens a PR in the demo repo (~10 s end-to-end).
4. The reviewer is redirected to Veridict’s review page. `mypy` + `pytest` + `Z3` run server-side; the green spec-check banner appears.
5. Tick “I’ve reviewed these changes,” click *Approve anonymously*. Prover runs (~10 s). Receipt page shows the circuit ID, proof size, transcript hex, prover/verifier timings, and pseudonym.
6. On GitHub, an anonymous comment appears under `@anonymous-review-bot[bot]` carrying the pseudonym, identicon, and reputation chips. The `zk-review-gate` commit status updates to pending: 1 / 2.
7. In a second browser session, repeat as a different reviewer. The status flips to success: 2 / 2 -- merge gate green. The *Merge* button on GitHub enables.

4.2 Quantitative observations

Metric	Value
Claude synthesis latency (end-to-end PR open)	~10 s
Spec-check latency (<code>mypy</code> + <code>pytest</code> + <code>Z3</code>)	< 3 s for a typical 3-file PR
Prover wall-clock time	~10 s (~8 s circuit compile, ~2 s proof)
Verifier wall-clock time	~7 s
Proof size on disk	361 408 bytes (~360 KB)
Verifier peak RSS	~365 MB

4.3 Failure modes observed and fixed

- **Claude generated TokenBucket(capacity=100, rate=0) test cases** that raised against the implementation's positive-rate precondition. Fixed by tightening the synthesis prompt to demand valid inputs in all non-raises tests.
- **mypy errored on from z3 import *** wildcard imports in the Z3 files (name-undefined). Fixed by running mypy only on implementation files, not test or Z3 files.
- **Verifier OOM-killed on a 512 MB instance.** Moved both services to Cloud Run with 4 GB.
- **Backend rejected proofs because the hardcoded claim value was "maintainer"** while regular org members got "reviewer". Fixed by trying both CBOR-encoded values on every verify call.

* * *

5. Discussion

Veridict's most useful insight is structural: the architectural hole in OSS review is *not* the absence of clever tooling, it is the lack of separation between *being a qualified reviewer* and *being a known identity*. Every additional gadget (linters, fuzzers, static analysers) gets attached to the same identity-bound approval, and so the identity stays the lightning rod.

The cleanest outcome is that the system makes anonymity *cheaper than identity-bound review*. A reviewer raising a security concern can sign anonymously without becoming a target. A maintainer overwhelmed by AI PRs can require N anonymous credentialed approvals rather than gating on their own time. Trust is distributed.

A subtler outcome: **the spec-check gate matters more than the ZK layer for the AI-synthesis use case**. The ZK layer protects reviewers; the spec-check gate protects the codebase from confidently-wrong AI output. Both layers are necessary, but the spec gate is what closes the loop between *the AI wrote it* and *we're shipping it*.

The Longfellow primitive itself is more capable than this prototype exercises. Longfellow was designed for selectively disclosing arbitrary MDOC attributes; we use it only to disclose a single role attribute. A richer credential schema (described in §5.1 below) could prove things like “*approved at least 10 prior PRs in this organisation*” or “*holds a domain badge in cryptography*” without disclosing the underlying audit log.

5.1 What would be done with more time

1. **Nullifier circuit.** Derive a deterministic blinded ID inside the ZK proof from (reviewer_secret, pr_key). Today a reviewer with two sessions can sign two approvals on one PR and they look like two pseudonyms. A nullifier collapses them.
2. **Browser-side proving.** Compile Longfellow's prover to WASM so the device key never leaves the reviewer's browser. The issuer becomes a pure attestation service.
3. **Inline review comments.** The bot already has pull_requests:write. Switch from top-level comments to per-line review comments via POST /pulls/{N}/reviews.

4. **Spec-in-credential.** Embed the hash of the spec the credential was issued against as a disclosed MDOC attribute. The proof then cryptographically binds “*this code satisfies spec X*” to “*a qualified reviewer approved it.*”
5. **Multi-language spec checking.** Extend the gate beyond Python: Dafny for correctness proofs, cargo test for Rust, property-based tests with Hypothesis.
6. **ZK reputation aggregation.** Move from coarse OAuth-derived chips (5y+ on GitHub) to ZK-proven facts (*this reviewer approved 10+ prior PRs in this org*).

* * *

Appendix: Limitations and Dual-Use Considerations

Limitations

- **No nullifier yet.** Two browser sessions → two pseudonyms for the same reviewer, defeating “N independent reviewers” semantics. The current deployment relies on operational checks (require sign-in, rate-limit per session) rather than cryptographic uniqueness.
- **Issuer-held device key.** The issuer mints a fresh device key per session for hackathon speed. A real deployment must generate the device key client-side; otherwise the issuer can impersonate any reviewer.
- **Spec coverage is Python-only.** mypy + pytest + Z3 only run on .py files. PRs in Go, Rust, C++ are passed through unverified (skipped, not blocked). This means an attacker could land a malicious patch in a non-Python file with no formal gate.
- **Z3 layer proves spec consistency, not implementation correctness.** The Z3 file asserts that the *specification* is internally consistent (each invariant’s negation is unsat for the spec’s symbolic model). It does not prove that the synthesised Python code implements the specification. Strengthening this requires a refinement step (e.g. Dafny) that the hackathon timebox did not allow.
- **Persistence is SQLite, not replicated.** Approval state lives in a SQLite database that is durable across restarts but is not replicated. Cloud Run’s stateless nature means scaled instances would diverge without a managed Postgres back-end.
- **Single bot identity on GitHub.** All anonymous reviewers post as @anonymous-review-bot[bot]. Pseudonyms + identicons inside the comment body distinguish them, but a hostile observer of the PR can correlate timing or comment style with sessions. A pool of bot accounts would be more robust at the cost of operational complexity.
- **Issuer is a single point of failure.** A compromised issuer can mint credentials at will. The hackathon prototype runs one issuer; a production system would need a federated issuer model with cross-issuer attestation.
- **Reputation chip leakage.** The chips (“5y+ on GitHub”, “100+ repos”) are coarse by design, but the joint distribution can narrow down identity in small organisations. For low-population orgs, this chip set should be reduced to role only.

Dual-Use Considerations

The same architecture that lets *defensive* reviewers raise concerns anonymously could be misused by hostile actors. We name the failure modes explicitly:

- **Approval laundering.** A hostile maintainer who controls the issuer (or who has compromised a small set of credentialed reviewers) could issue anonymous approvals to themselves and

pass arbitrary code. Mitigation: enforce that issuers cannot also be reviewers, and require the credential's OAuth-derived chips to disagree across approvals (e.g. role mix, account-age bucket mix).

- **Spec-gate gaming.** An attacker who controls the synthesis step (or simply writes the PR by hand) can craft code that passes mypy + pytest + Z3 trivially while shipping a side-channel or supply-chain attack (e.g. a `setup.py` that does the malicious work, or a runtime dependency added in `requirements.txt`). The spec gate is necessary but not sufficient; it does not replace dependency auditing, sandboxing, or secret-scanning.
- **Identity laundering.** A maintainer with a tarnished reputation could acquire a credential and re-enter the review process anonymously. This is intentional in the threat model (the system is supposed to decouple reputation from identity), but in a project where past misbehaviour matters, the operator must decide whether identity persistence is desirable. A nullifier combined with per-org allow-listing would address this for cases where it does matter.
- **Anonymous harassment via approval flood.** A reviewer with valid credentials but malicious intent could approve all PRs to manufacture consent. Mitigation: rate-limit per credential, and require that the approval comment carry substantive content (Veridict already allows a free-form reviewer comment; making it mandatory would close the loophole).
- **Credential exfiltration.** If a reviewer's session token is stolen, the attacker can prove ownership without holding the underlying GitHub identity. Mitigation: short MDOC validity windows (currently 10 minutes), per-PR binding, and rotate the issuer signing key on suspicion.

The system is honestly described as *anonymous toward the verifier* (CI, the PR author, the audit trail) and *not anonymous toward the issuer*. A reviewer who only trusts their own browser would not be protected by the current architecture; they would need the browser-side proving extension.

* * *

References

Bincode development has ceased permanently. r/rust, December 2024. https://www.reddit.com/r/rust/comments/1pnz1iz/bincode_development_has_ceased_permanently/

XZ Utils backdoor (CVE-2024-3094). Wikipedia, retrieved May 2026. https://en.wikipedia.org/wiki/XZ_Utils_backdoor

Stenberg, Daniel. *The I in LLM stands for intelligence.* Daniel.haxx.se blog, January 2024.

Longfellow Zero-Knowledge: Google's ZK system for ISO 18013-5 MDOC credentials. Dyne.org analysis, 2024. <https://news.dyne.org/longfellow-zero-knowledge-google-zk/>. Reference implementation: <https://github.com/google/longfellow-zk>.

de Moura, Leonardo, and Bjorner, Nikolaj. *Z3: An Efficient SMT Solver.* TACAS 2008.

ISO/IEC 18013-5:2021. *Personal identification, ISO-compliant driving licence, Part 5: Mobile driving licence (mDL) application.*

Apart Research. *Secure Program Synthesis Hackathon, May 22–24, 2026.* <https://apartresearch.com/sprints/secure-program-synthesis-hackathon-2026-05-22-to-2026-05-24>